

(Task 1 of 20) In this tutorial, we will learn *more* about **mutable values**, illustrated with **arrays**.

0

(Task 2 of 20) What is the result of running this program?

Python | Python [Run 

Lispy

```
x = [ 74, 82 ]    (defvar x (mvec 74 82))
y = [ x ]        (defvar y (mvec x))
x[0] = y         (vec-set! x 0 y)
print(x[1])      (vec-ref x 1)
```

error

2

The answer is 82.

3

Textual explanation

You might think x and y can't form a circle. However, this is not the case. In SMoL, arrays can be referred to by arbitrary arrays, including themselves.

Click [here](#) to run this program in the Stacker.

What is the result of running this program?

Python | Python [Run 

Lispy

```
a = [ 41, 92 ]    (defvar a (mvec 41 92))
b = [ a ]        (defvar b (mvec a))
a[1] = b         (vec-set! a 1 b)
print(a[0])      (vec-ref a 0)
```

41

5

You predicted the output correctly 🎉🎉🎉

6

(Task 3 of 20) What is the result of running this program?

Python | 

Python [Run]

Pseudo

```
x = [ 62 ]   let x = vec[62]
y = x       let y = x
y[0] = 34   y[0] = 34
print(x)    print(x)
```

[34]

8

Please briefly explain why you think the answer is [34].

9

y mutated it into [34]

10

You predicted the output correctly 🎉🎉🎉

11

x is first bound to a array that refers 62. y is bound to the same array. The array mutation replaces 62 with 34. So, eventually, the array becomes [34].

Click [here](#) to run this program in the Stacker.

(Task 4 of 20) What is the result of running this program?

Python | 12

Python [Run]

Pseudo

```
x = [ 52, 70, 61 ]   let x = vec[52, 70, 61]
y = x               let y = x
x[0] = 93           x[0] = 93
print(y[0])         print(y[0])
```

93

13

You predicted the output correctly 🎉🎉🎉

14

The program creates a array where the 0-th element is 52. and binds it to both x and y. The 0-th element of the array is then replaced with 93. So, the result is 93.

Click [here](#) to run this program in the Stacker.

(Task 5 of 20) What is the result of running this program?

Python | 15

Python [Run]

JavaScript

```

x = [ 99, 83 ]
def f(y):
    x[0] = 34
    return y
print(f(x))

let x = [ 99, 83 ];
function f(y) {
    x[0] = 34;
    return y;
}
console.log(f(x));

```

[34, 83]

16

You predicted the output correctly 🎉🎉🎉

17

The program creates an array and binds it to `x`. The function call `f(x)` binds `y` to the same array. The function replaces the 0-th element of the array with 34 and then returns the array. So, the final result is the array, which is printed as `[34, 83]`.

Click [here](#) to run this program in the Stacker.

Python | 18

(Task 6 of 20) What is the result of running this program?

Python [Run]

Lispy

```

x = [ 43, 54 ]
x[0] = x
print(x[1])

(defvar x (mvec 43 54))
(vec-set! x 0 x)
(vec-ref x 1)

```

54

19

You predicted the output correctly 🎉🎉🎉

20

`x` is bound to an array. `x[0] = x` makes the array refer to itself. This is fine because an array element can be any value, including itself. Besides, the array is not copied, so the mutation finishes immediately.

Click [here](#) to run this program in the Stacker.

Python | 21

(Task 7 of 20) What is the result of running this program?

(Task 7 of 20) What is the result of running this program?

Python [Run]

Scala 3

```

x = [ 19, 73, 28 ]   val x = Buffer(19, 73, 28)
y = [ x, x ]         val y = Buffer(x, x)
y[0][0] = 64         y(0)(0) = 64
print(y[1])          println(y(1))

```

[19, 73, 28]

22

Please briefly explain why you think the answer is [19, 73, 28].

23

the first list first element was mutated not the seconds list first element

24

The answer is [64, 73, 28].

25

Textual explanation

You might think the two elements of `x` refer to different arrays, so changing one doesn't change the other. However, they refer to the same array. In SMoL, a array can be referred to by multiple places, and creating new arrays that refer to existing ones does not create new copies of the existing arrays.

Click [here](#) to run this program in the Stacker.

Graphical explanation

You might also want to check [a graphical explanation](#)

What is the result of running this program?

Python [Run]

20

Python [Run]

Lispy

```

mv = [ 63 ]           (defvar mv (mvec 63))
mv2 = [ mv, mv ]      (defvar mv2 (mvec mv mv))
mv2[0][0] = 42        (vec-set! (vec-ref mv2 0) 0 42)
print(mv2[1])         (vec-ref mv2 1)

```

42

27

You predicted the output correctly 🎉🎉🎉

28

(Task 8 of 20) What is the result of running this program?

Python [Run]

29

Python [Run]

Scala 3

```

x = [ 10, 48, 95 ]
def f(y):
    y[0] = 32
    return y[0]
z = f(x)
print(x[0])

val x = Buffer(10, 48, 95)
def f(y : Int) =
    y(0) = 32
    y(0)
val z = f(x)
println(x(0))

```

32

30

You predicted the output correctly 🎉🎉🎉

31

The program first creates a three-element array, where the 0-th element is 10. After that, the program defines a function f. The function call f(x) replaces the 0-th array element with 32. After that, the array is printed. So the result is 32.

Click [here](#) to run this program in the Stacker.

(Task 9 of 20) What is the result of running this program?

Python [Run]

Python [Run]

JavaScript

```

x = [ 53 ]
v = [ 72, x ]
x[0] = 72
print(v)

let x = [ 53 ];
let v = [ 72, x ];
x[0] = 72;
console.log(v);

```

[72, [72]]

33

You predicted the output correctly 🎉🎉🎉

34

x is bound to a one-element array. The only element of this array is 53. v is bound to a two-element array. The elements of this array are 72 and the one-element array. The subsequent x[0] = 72 mutates the one-element array. Finally, the value of v is printed.

Click [here](#) to run this program in the Stacker.

(Task 10 of 20) What did you learn about arrays from these programs?

35

arrays can be mutated different ways

36

(Task 11 of 20) A array can be referred to by more than one variable and even by other arrays (including itself). Referring to a array does not create a copy of the array; rather, they share the same array. Specifically

37

- Binding a array to a new variable does not create a copy of that array.
- Arrays that are passed to a function in a function call do not get copied.
- Creating new arrays that refer to existing ones does not create new copies of the existing arrays.

The references share the same array. That is, arrays can be **aliased**.

Any feedback regarding these statements? Feel free to skip this question.

38

(You skipped the question.)

39

(Task 12 of 20) Now please scroll back and select 1-3 programs that make the above points.

40

You don't need to select *all* such programs.

(You selected 3 programs)

41

Okay. How do these programs (1,4,7) support the point?

42

They bind arrays to a new variable

43

(Task 13 of 20) Reconsider these two programs that you might have seen. The only difference is in `y = ____`.

Python | Python [Run ]

Pseudo

```
x = [ 52, 96 ]   let x = vec[52, 96]
y = x            let y = x
x[0] = 34        x[0] = 34
print(y)         print(y)
```

Python [Run]

Pseudo

```

x = [ 52, 96 ]    let x = vec[52, 96]
y = [ 52, 96 ]    let y = vec[52, 96]
x[0] = 34         x[0] = 34
print(y)         print(y)

```

It is tempting to describe the variables as

- x is bound to [52, 96]
- y is bound to [52, 96]

This description does not help us to understand the program because it can't explain why `x[0] = 34` mutates y in one case, and does not in the other.

What is a better way to describe the bindings that help solve this problem?

since `y = x` and `x` was mutated then `y` also mutates

45

We can say, in the first program

46

- x is bound to @100
- y is bound to @100

where

- @100 is [52, 96]

While for the other program

- x is bound to @100
- y is bound to @200

where

- @100 is [52, 96]
- @200 is [52, 96]

(Task 14 of 20) In SMoL, each array has its own unique **heap address** (e.g., @100 and @200). The mapping from addresses to arrays is called the **heap**.

47

(**Note:** we use @ddd (e.g., @123, @200, and @100) to represent heap addresses. Heap addresses are *random*. The numbers don't mean anything.)

Any feedback regarding these statements? Feel free to skip this question.

48

(You skipped the question.)

49

(Task 15 of 20) Which choice best describes the status of the heap at the end of the following program?

Python | Python [Run 

Lispy

```
x = 3                (defvar x 3)
v = [ 1, 2, x ]      (defvar v (mvec 1 2 x))
```

51

- A. @1 = #(1 2 3)
- B. @1 = #(1 2 x)
- C. There is nothing in the heap.

@1 = #(1 2 3)

52

You gave the correct answer 🎉🎉🎉

53

C is wrong because the [1, 2, x] creates a array. Every array is stored on the heap.

B is wrong because arrays refer values, while x is not a value.

(Task 16 of 20) Which choice best describes the status of the heap at the end of the following program?

Python | Python [Run 

Scala 3

```
mv = [ 3 ]           val mv = Buffer(3)
mv2 = [ mv, mv ]      val mv2 = Buffer(mv, mv)
mv2[0][0] = 42        mv2(0)(0) = 42
```

55

- A. @100 = #(3); @200 = #(@100 @100)
- B. @100 = #(3); @200 = #(@300 @100); @300 = #(42)
- C. @100 = #(3); @200 = #(#(42) #(3))
- D. @100 = #(42); @200 = #(@100 @100)
- E. @100 = #(42); @200 = #(#(42) #(42))
- F. There is nothing in the heap.

@100 = #(3); @200 = #(#(42) #(3))

56

The answer is @100 = #(42); @200 = #(@100 @100).

57

E and **C** are wrong. Arrays refer values. [42] and [3] are not values, although some array values are printed like them. So, @200 = #(#(42) #(42)) and @200 = #(#(42) #(3)) can not be valid.

[3] creates a 1-element array, @100. The only element is 3. [mv, mv] creates a 2-element array, @200. Both elements of @200 are @100. No more arrays are created, which means **B** must be wrong. So, then, the correct answer must be **A** or **D**.

However, the subsequent mutation changes @100 (the first element of @200). The 0-th element of @100 is mutated to 42. So, **D** is the correct answer.

Python | 

(Task 17 of 20) The following program defines two variables but creates nothing on the heap.

Python [Run 

Lispy

```
x = 2      (defvar x 2)
y = 3      (defvar y 3)
print(x)   x
print(y)   y
```

The following program defines no variables but creates two things on the heap.

Python [Run 

Lispy

```
print([ 1, [ 2, 3 ] ]) (mvec 1 (mvec 2 3))
```

What did you learn from this pair of programs?

heaps can use heap address in themselves to refer to them

59

(Task 18 of 20)

60

- Creating a array does not inherently create a binding.
- Creating a binding does not necessarily alter the heap.

Any feedback regarding these statements? Feel free to skip this question.

61

(You skipped the question.)

62

(Task 19 of 20) Here is a program that confused many students

Python | 

Python 

Pseudo

```
v = [ 1, 2, 3, 4 ]   let v = vec[1, 2, 3, 4]
vv = [ v, v ]       let vv = vec[v, v]
vv[1][0] = 100      vv[1][0] = 100
print(vv)           print(vv)
```

Please

1. Run this program in the stacker by clicking one of the  buttons above;
2. The stacker would show how this program produces its result(s);
3. Keep clicking  until you reach a configuration that you find particularly helpful;
4. Click  [Share This Configuration](#) to get a link to your configuration;
5. Submit your link below;

<https://smol-tutor.xyz/stacker/?syntax=Python&randomSeed=smol-tutor&hole=%E2%80%A2&nNext=2&program=%0A%28defvar+v+%28mvec+1+2+3+4%29%set%21+%28vec-ref+vv+1%29+0+100%29%0Avv%0A&readOnlyMode=>

64

(Task 20 of 20) Please write a couple of sentences to explain how your configuration explains the result(s) of the program.

65

it shows the results and how it is stored in memeory

66

Let's review what we have learned in this tutorial.

67

A array can be referred to by more than one variable and even by other arrays (including itself). Referring to a array does not create a copy of the array; rather, they share the same array. Specifically

- Binding a array to a new variable does not create a copy of that array.
- Arrays that are passed to a function in a function call do not get copied.
- Creating new arrays that refer to existing ones does not create new copies of the existing arrays.

The references share the same array. That is, arrays can be **aliased**.

In SMoL, each array has its own unique **heap address** (e.g., @100 and @200). The mapping from addresses to arrays is called the **heap**.

(**Note:** we use @ddd (e.g., @123, @200, and @100) to represent heap addresses. Heap addresses are *random*. The numbers don't mean anything.)

- Creating a array does not inherently create a binding.
- Creating a binding does not necessarily alter the heap.

You have finished this tutorial 🎉🎉🎉

Please print the finished tutorial to a PDF file so you can review the content in the future. **Your instructor (if any) might require you to submit the PDF.**

Start time: 1760133758169